

linear_chain2.py: сценарий линейной цепи

математика, реализация и обоснование методов

Документ сгенерирован по исходному коду симулятора

Аннотация

Модуль `scenarios/linear_chain2.py` реализует формирование линейной цепочки БПЛА в горизонтальной плоскости: два крайних дрона (якоря) стремятся к фиксированным концам отрезка AB , внутренние агенты используют *гибридный* закон управления — геометрическую стабилизацию порядка на линии и распределённый закон из постановки «коридор / `corridor_sweeping`» (четыре полупространства, нелинейность $\tanh$, виртуальный зазор). Управление низкоуровневое: переопределение каналов RC (roll/pitch) при режиме удержания позиции в SITL. Ниже приведены математический аппарат, соответствие функциям файла и мотивы инженерных решений.

Содержание

1	Назначение и контекст	3
1.1	Задача	3
1.2	Ограничения платформы	3
1.3	Связь с <code>linear_chain.py</code> (версия 1)	3
2	Архитектура программы	3
2.1	Потоки выполнения	3
2.2	Точка входа	3
3	Системы координат	3
3.1	Общий кадр (<code>common frame</code>)	3
3.2	Отрезок AB и базис цепи	4
4	Математический аппарат внутреннего контура	4
4.1	Множество соседей в радиусе видимости	4
4.2	Четыре полупространства и расстояния $d_{a\pm}, d_{c\pm}$	4
4.3	Нелинейность и замена «бесконечности»	4
4.4	Сигналы σ_a, σ_c	4
4.5	Геометрические ошибки равномерности	5
4.6	Комбинированный вектор в NED	5
5	От s_{xy} к командам RC	5
5.1	Поворот NED $\rightarrow$ корпус (вперёд / вправо)	5
5.2	Демпфирование по скорости	5
5.3	Пропорциональный закон, bang-bang минимум и slew	5
5.4	Маппинг на каналы	6
6	Якоря: PID к точке	6
7	Соответствие: функции файла и роль	6

8	Обоснование выбора методов	6
9	Параметры командной строки и константы	7

1 Назначение и контекст

1.1 Задача

Сконфигурировать рой из $N \geq 4$ дронов так, чтобы в установившемся режиме они выстроились **вдоль заданного отрезка** с закреплёнными концами (якоря с минимальным и максимальным идентификатором) и относительно равномерным распределением внутренних звеньев.

1.2 Ограничения платформы

В симуляторе ArduPilot SITL доступны не произвольные силы в инерциальном базисе, а **псевдо-удержание** через режим позиционного удержания и `MAV_OVERRIDE` по каналам наклона. Поэтому закон формируется как желаемый *горизонтальный* вектор «куда толкнуть-ся», затем проецируется на оси корпуса и переводится в приращения PWM.

1.3 Связь с `linear_chain.py` (версия 1)

В `linear_chain` внутренний дрон строит сглаженную целевую точку на отрезке (середина между соседями по проекции s) и идёт к ней через PID. В `linear_chain2` внутренний контур **не** задаёт явную 2D-цель; вместо этого суммируются два вклада в плоскости NED и далее применяется пропорциональный закон по стикам с ограничениями и демпфированием скорости.

2 Архитектура программы

2.1 Потоки выполнения

1. **Инициализация** (`initialize_drone_parallel`): для каждого дрона — подключение, сценарий `INIT_STEPS` (режим, arm, взлёт, `POS_HOLD`, нейтральный RC, стримы позиции и attitude, keepalive).
2. **Обмен координатами** (`coordinate_exchange_loop`): с частотой `-exchange-hz` собираются позиции всех дронов в *общем* кадре и рассылаются остальным через `update_other_drone_position`. Опционально — публикация во внешний визуализатор.
3. **Поведение якорей** (`endpoint_loop`): PID в корпусе к точкам A или B .
4. **Поведение внутренних** (`internal_chain_anatoliy_loop`): гибридный вектор $\mathbf{c}_{xy}$ и P-закон по RC.

2.2 Точка входа

Функция `main()` парсит аргументы (`-drones`, `-duration`, `-segment-length`, `-exchange-hz`, `-r-vis`, `-w-cross`), создаёт контроллеры, барьер синхронизации и запускает перечисленные потоки после паузы и фиксации концов отрезка по текущим позициям якорей.

3 Системы координат

3.1 Общий кадр (`common frame`)

Локальные координаты разных экземпляров SITL смещены. Для согласованного обмена вводится сдвиг по оси, соответствующей East в NED (в коде поле `y`):

$$y_{\text{common}} = y_{\text{local}} + (\text{id} - 1) \cdot 2 \text{ м.} \quad (1)$$

Реализация: `_did_offset_y`, `_my_position_common`.

3.2 Отрезок AB и базис цепи

После взлёта концы отрезка задаются в common frame. По оси «вдоль заданной геометрии симулятора» (в коде это сдвиг по x на $\pm L/2$) и с **общей** ординатой

$$y_{\text{mid}} = \frac{y_{\text{left}} + y_{\text{right}}}{2}, \quad A = \left(-\frac{L}{2}, y_{\text{mid}}\right), \quad B = \left(+\frac{L}{2}, y_{\text{mid}}\right). \quad (2)$$

Единичный вектор вдоль цепи и левый перпендикуляр в плоскости XY :

$$\mathbf{u} = \frac{B_{xy} - A_{xy}}{\|B_{xy} - A_{xy}\|}, \quad \mathbf{n} = (-u_y, u_x)^\top. \quad (3)$$

Проекция произвольной точки p на ось от A вдоль $\mathbf{u}$:

$$s(p) = (p_{xy} - A_{xy}) \cdot \mathbf{u}. \quad (4)$$

Функции: `_segment_endpoints_common_from_anchors`, `_unit_vector_xy`, `_normal_left_xy`, `_projection_s`.

4 Математический аппарат внутреннего контура

4.1 Множество соседей в радиусе видимости

Для каждого другого дрона j внутри радиуса R_{vis} (`R_VIS_M`) относительно своей позиции p_{me} :

$$\Delta_j = p_{j,xy} - p_{\text{me},xy}, \quad a_j = \Delta_j \cdot \mathbf{u}, \quad c_j = \Delta_j \cdot \mathbf{n}. \quad (5)$$

Требование `len(peers) ≥ 2` отсекает вырожденные случаи до появления достаточного числа наблюдений.

4.2 Четыре полупространства и расстояния $d_{a\pm}$, $d_{c\pm}$

По списку пар (a_j, c_j) вычисляются минимальные положительные смещения в каждом квадранте осей (`along`, `cross`) — аналог `get_nearest_distances` в `corridor_sweeping`:

$$\begin{aligned} d_{a+} &= \min\{a_j : a_j \geq 0\}, & d_{a-} &= \min\{|a_j| : a_j < 0\}, \\ d_{c+} &= \min\{c_j : c_j \geq 0\}, & d_{c-} &= \min\{|c_j| : c_j < 0\}, \end{aligned}$$

с флагами наличия соседа в каждом направлении. Реализация: `_nearest_along_cross`.

4.3 Нелинейность и замена «бесконечности»

$$\xi(x) = \tanh(x), \quad g(d, w, f) = \begin{cases} d, & f = \text{истина (сосед есть)}, \\ w, & f = \text{ложь}. \end{cases} \quad (6)$$

По оси вдоль для отсутствующего направления подставляется $w = 0$; по поперёк — $w = w_{\text{cross}}$ (`W_CROSS_M`), имитируя виртуальную «стену» коридора.

4.4 Сигналы σ_a , σ_c

В полной постановке коридора в σ входят проекции скорости v_{along} , v_{cross} . В файле явно задано $v_{\text{along}} = v_{\text{cross}} = 0$ для устойчивости в SITL. Тогда

$$\sigma_a = -\xi(g(d_{a+}, 0, f_{a+})) + \xi(g(d_{a-}, 0, f_{a-})), \quad (7)$$

$$\sigma_c = -\xi(g(d_{c+}, w_{\text{cross}}, f_{c+})) + \xi(g(d_{c-}, w_{\text{cross}}, f_{c-})). \quad (8)$$

Реализация: `_xi`, `_g`, `_sigmas_segment_frame`.

Интерпретация: σ_a и σ_c — скалярные «силы» распределённого закона, согласующие близость к ближайшим соседям по четырём направлениям с насыщением; виртуальный зазор не даёт вектору обнуляться, если с одной стороны по поперёк нет пира.

4.5 Геометрические ошибки равномерности

Все агенты с известными позициями сортируются по $(s(p_i), \text{id})$. Для внутреннего с индексами соседей слева и справа:

$$e_{\text{along}} = \frac{s_L + s_R}{2} - s_{\text{me}} \quad (\text{если оба соседа есть; иначе } 0). \quad (9)$$

Поперечное отклонение от прямой через A вдоль $\mathbf{u}$:

$$e_{\text{cross}} = (p_{\text{me},xy} - A_{xy}) \cdot \mathbf{n}. \quad (10)$$

Ошибки ограничиваются по модулю константами `SPACING_E*_CLAMP_M`. Реализация: `_spacing_errors_pro`

4.6 Комбинированный вектор в NED

Геометрическая составляющая (главный вклад):

$$\mathbf{g}_{xy} = k_{\text{along}} e_{\text{along}} \mathbf{u} - k_{\text{cross}} e_{\text{cross}} \mathbf{n}. \quad (11)$$

Вклад закона Anatoliy:

$$\mathbf{a}_{xy} = \sigma_a \mathbf{u} + \sigma_c \mathbf{n}. \quad (12)$$

Итог ($w_A = \text{ANATOLIY_COMB_WEIGHT}$):

$$\mathbf{c}_{xy} = \mathbf{g}_{xy} + w_A \mathbf{a}_{xy}. \quad (13)$$

Мёртвые зоны: если нет «натяжения» по $e_{\text{along}}, e_{\text{cross}}$ вне `SPACING_E_DEADBAND_M` и $|\sigma_a|, |\sigma_c|$ ниже `SIGMA_DEADBAND`, отправляется нейтральный RC.

5 От $\mathbf{c}_{xy}$ к командам RC

5.1 Поворот NED $\rightarrow$ корпус (вперёд / вправо)

Пусть ψ — рыскание из attitude с учётом `YAW_SIGN`. Матрица совпадает с `_ned_to_body_forward_right`:

$$\begin{pmatrix} f_0 \\ r_0 \end{pmatrix} = \begin{pmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{pmatrix} \begin{pmatrix} c_N \\ c_E \end{pmatrix}, \quad (14)$$

где $(c_N, c_E)^\top$ — компоненты $\mathbf{c}_{xy}$ (North, East).

5.2 Демпфирование по скорости

Скорость (v_x, v_y) в NED переводится в (v_f, v_r) той же матрицей. Далее

$$f = f_0 - \text{clamp}(k_d v_f), \quad r = r_0 - \text{clamp}(k_d v_r), \quad (15)$$

где $k_d = \text{INTERNAL_BODY_V_DAMP}$, `clamp` ограничивает вклад по модулю `INTERNAL\_BODY\_V\_DAMP\_CLAMP`. Цель — уменьшить команды, если аппарат уже движется в ту же сторону в корпусных осях.

5.3 Пропорциональный закон, bang-bang минимум и slew

Целевые приращения PWM по осям:

$$\Delta_{\text{pitch}}^{\text{tgt}} = \text{PWM_axis}(f), \quad \Delta_{\text{roll}}^{\text{tgt}} = \text{PWM_axis}(r; k_p \cdot \text{INTERNAL_RC_KP_ROLL_MULT}), \quad (16)$$

где `PWM_axis` — округление $k_p \cdot$ ошибка с ограничением по модулю `INTERNAL\_RC\_MAX\_PWM`; если после округления получился ноль при ненулевой ошибке, выставляется минимальный шаг знака `INTERNAL\_RC\_MIN\_STEP\_PWM` (избегание залипания на нейтрале). За один такт приращение ограничивается `INTERNAL\_RC\_SLEW\_PWM\_PER\_CYCLE` (`_slew_int`).

5.4 Мэппинг на каналы

$$\text{pitch}_{\text{PWM}} = \text{RC_NEUTRAL} - \Delta_{\text{pitch}}, \quad \text{roll}_{\text{PWM}} = \text{RC_NEUTRAL} + \Delta_{\text{roll}}, \quad (17)$$

с опцией обмена каналов RC_OVERRIDE_SWAP_ROLL_PITCH под пользовательский RCMAP.

6 Якоря: PID к точке

Ошибка (e_N, e_E) в NED с deadband ERROR_DEADBAND_M переводится в (e_f, e_r) ; далее

$$u_{\text{pitch}} = \text{PID}_{\text{pitch}}(e_f), \quad u_{\text{roll}} = \text{PID}_{\text{roll}}(e_r), \quad (18)$$

с разными K_p (ENDPOINT_X_KP на канал pitch через forward, ENDPOINT_Y_KP на roll) и общим ограничением ENDPOINT_OUTPUT_LIMIT. После входа в радиус ENDPOINT_REACHED_THRESH_M — нейтральный удерживающий RC.

7 Соответствие: функции файла и роль

_segment_endpoints_common_from_anchors	Точки A, B с общим y_{mid}
_unit_vector_xy, _normal_left_xy	Базис $(\mathbf{u}, \mathbf{n})$
_nearest_along_cross	$d_{a\pm}, d_{c\pm}$ и флаги
_sigmas_segment_frame	σ_a, σ_c
_spacing_errors_projection	$e_{\text{along}}, e_{\text{cross}}$
_pwm_axis, _slew_int	P-закон, минимальный шаг, slew
_move_towards_common_with_pid	Якорный контур
coordinate_exchange_loop	Рассылка позиций

8 Обоснование выбора методов

Локальный базис $(\mathbf{u}, \mathbf{n})$ вместо глобальных осей. Закон коридора изначально формулируется в осях «вдоль / поперёк» границы. Привязка к отрезку AB делает цепь **инвариантной к наклону** линии в плоскости (как и в linear_chain).

Среднее y_{mid} для обоих концов AB . Раздельные y левого и правого якоря после взлёта в SITL часто расходятся из-за шума и сдвига снимка; тогда целевая прямая наклонена, внутренние «зависают» на разных поперечных уровнях (ступенька на 2D-графике). Общее y_{mid} **жёстко задаёт одну прямую** для геометрической части.

Гибрид $\mathbf{g}_{xy} + w_A \mathbf{a}_{xy}$. Чистый закон коридора сильно влияет на распределение вокруг локальных препятствий-соседей; для *ровной линии* доминирует геометрия середины между соседями по s . Малый w_A (ANATOLIY_COMB_WEIGHT) даёт **мягкую нелинейную поправку** без разрушения выравнивания на AB (комментарий в коде: иначе мешает «лечь» на одну прямую).

$\tanh$ в σ . Ограничивает чувствительность на малых расстояниях и предотвращает неограниченный рост вклада при сближении — типичный приём в распределённых законах с «мягким» отталкиванием/притяжением.

Виртуальный w_{cross} . Если с одной стороны поперёк нет видимого соседа, подстановка конечного зазора в g сохраняет ненулевой градиент σ_c — аналог стенки коридора в исходной постановке.

$v = 0$ в σ в SITL. Поток скорости из MAVLink часто зашумлён или не гарантирован в нужном базисе; ненулевые $v_{\text{along}}, v_{\text{cross}}$ приводили бы к **дрейфу** σ . Отключение — компромисс: статическая нелинейность по положению соседей вместо полной динамической модели point-mass.

RC override вместо waypoint-миссии. Единый интерфейс с симулятором и возможность тонко настраивать отклик в режиме удержания; минимальный шаг PWM и slew снижают перерегулирование и рывки на дискретной сетке CONTROL_HZ.

Поворот ошибки в корпус перед PID (якоря) и перед P (внутренние). Команды roll/pitch действуют в плоскости корпуса; без учёта ψ при ненулевом рыскании вектор усилий в NED искажается.

YAW_SIGN и swap roll/pitch. Согласование знака yaw SITL ArduPilot с ожидаемой ориентацией NED→body; swap компенсирует несовпадение RCMAP у пользователя.

9 Параметры командной строки и константы

CLI: -drones, -duration, -segment-length, -exchange-hz, -r-vis, -w-cross. Основные константы см. в начале linear_chain2.py и в Markdown linear_chain2_control.md.

Заключение

Файл linear_chain2.py реализует многоагентную линейную конфигурацию с разделением ролей (якоря / внутренние), явным математическим разделением геометрической стабилизации на отрезке и распределённого закона «коридора», адаптированного под ограничения RC-управления в SITL. Документ можно включать в отчёт как приложение; актуальность формул следует сверять с версией исходного кода.